1. Aim of the practical: Authenticate the student who is checking marks from university

portal as created in experiment

2. Objective: To enhance the university course application by introducing a secure student login system that allows authenticated students to view their marks. This will involve setting up user authentication using JWT (JSON Web Token) and password hashing for secure login.

3. Algorithm/Flowchart:

A) App.js :

```
import { BrowserRouter, Routes, Route, Link, useNavigate } from "react-router-dom";
import Register from "./pages/Register";
import Login from "./pages/Login";
import Marks from "./pages/Marks";
import { useEffect, useState } from "react";

function App() {
  const [isLoggedIn, setIsLoggedIn] = useState(false);
  const navigate = useNavigate();

  useEffect(() => {
    const token = localStorage.getItem("token");
    setIsLoggedIn(!!token);
  }, []);

  const handleLogout = () => {
    localStorage.removeItem("token");
    setIsLoggedIn(false);
    alert("You have logged out successfully!");
    navigate("/login");
  };
```

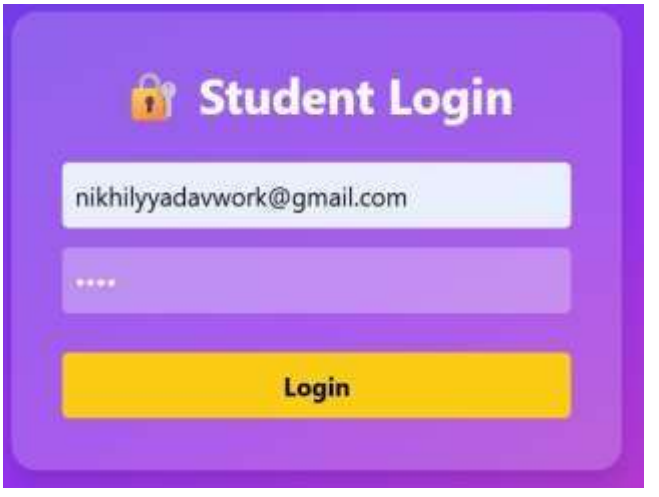
```
  return (
    <div className="min-h-screen bg-gradient-to-br from-indigo-600 via-purple-600 to-pink-500 text-
      white">
      <nav className="flex justify-center gap-8 bg-white/20 backdrop-blur-md py-4 text-lg font-semibold
      shadow-md">
        {!isLoggedIn && (
          <>
            <Link className="hover:text-yellow-300 transition" to="/">Register</Link>
            <Link className="hover:text-yellow-300 transition" to="/login">Login</Link>
          </>
        )}
        {isLoggedIn && (
          <>
            <Link className="hover:text-yellow-300 transition" to="/marks">Marks</Link>
            <button
              onClick={handleLogout}
              className="hover:text-red-400 transition font-bold"
            >
              Logout
            </button>
          </>
        )}
      </nav>

      <div className="flex justify-center items-center pt-12">
        <Routes>
          <Route path="/" element={<Register />} />
          <Route path="/login" element={<Login setIsLoggedIn={setIsLoggedIn} />} />
          <Route path="/marks" element={<Marks />} />
        </Routes>
      </div>
    </div>
  );
}

export default function AppWrapper() {
  return (
    <BrowserRouter>
      <App />
    </BrowserRouter>
  );
}
```

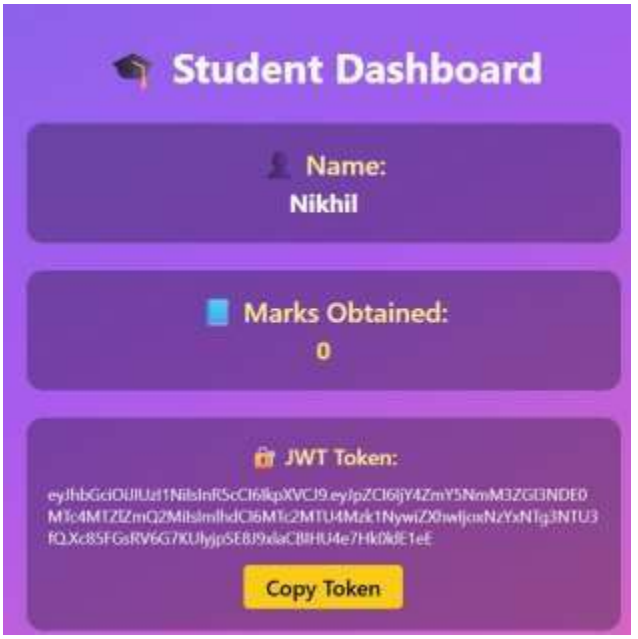
B) Server.js:

```
// server.js
const express = require("express");
```

```
const mongoose = require("mongoose");
const dotenv = require("dotenv");
const cors = require("cors");
const studentRoutes = require("./routes/studentRoutes");

dotenv.config();

const app = express();
app.use(express.json());
app.use(cors());

app.use("/api/student", studentRoutes);

mongoose.connect(process.env.MONGO_URI, {
  useNewUrlParser: true,
  useUnifiedTopology: true,
})
  .then(() => console.log(" MongoDB Connected"))
  .catch(err => console.error("+ MongoDB connection failed:", err.message));

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

4. Result/Output:





Learning outcomes (What I have learnt):

- Applied JWT-based session management to protect sensitive routes.
- Gained hands-on experience in building secure login, registration, and authorization workflows.
- Understood real-world concepts like token expiration, protected routes, and user state management.